

PROJET AWS - Clustering ECS / Kubernetes

Keysan BAKARI

Mastère Cybersécurité & Cloud Computing – Promotion MCS 26.4

IPSSI – Amazon AWS : ECS & EKS

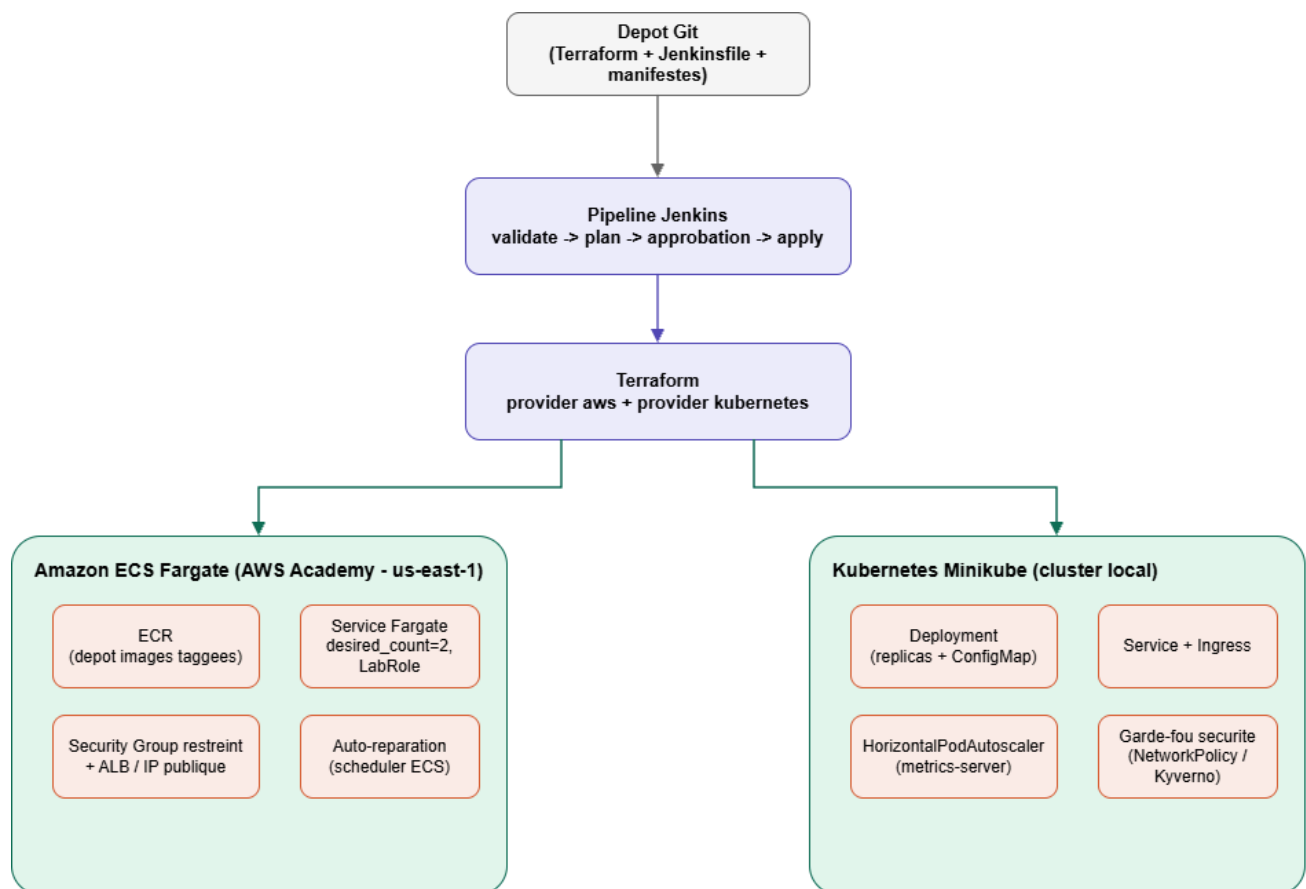
1. Contexte et objectif

Le but de ce projet était de déployer une même application sur deux orchestrateurs différents (Amazon ECS et Kubernetes) et d'automatiser les deux déploiements avec Terraform et Jenkins. L'idée derrière ça : dans une vraie boîte, on a souvent une partie de l'infra en cloud managé (ECS pour la prod) et une autre plus portable qui doit pouvoir tourner ailleurs (Kubernetes). Le but n'était pas de comparer les deux mais de montrer qu'on est capable d'industrialiser le déploiement sur les deux en même temps, avec une seule chaîne d'automatisation.

Contraintes imposées par le sujet : côté ECS, on devait rester sur AWS Academy (région us-east-1, rôle LabRole imposé, pas de création de rôle IAM). Côté Kubernetes, EKS n'étant pas disponible sur Academy, on est passé par Minikube en local sur ma VM Kali.

2. Architecture mise en place

Le schéma ci-dessous résume le flux global : le dépôt Git contient tout le code (Terraform des deux cibles + Jenkinsfile), Jenkins récupère ce code et pilote Terraform, qui déploie en parallèle sur ECS Fargate et sur le cluster Kubernetes Minikube.



Legende: gris = versioning, violet = automatisation, vert = plateforme cible, corail = ressource deployee

J'ai choisi une appli très simple (nginx avec une page HTML personnalisée) pour me concentrer sur l'infra plutôt que sur le développement applicatif, ce qui n'était de toute façon pas l'objectif du TP.

3. Déploiement sur Amazon ECS

Le module Terraform ecs (dossier modules/ecs) crée : un dépôt ECR pour stocker l'image, un cluster ECS Fargate, une task definition qui utilise le rôle LabRole imposé par AWS Academy comme execution role, un service avec 2 tâches désirées, et un Security Group restreint au port 80.

3.1 Choix techniques

- desired_count = 2 : ça permet de démontrer le scaling et l'auto-réparation du scheduler ECS (si une tâche crashe, ECS en relance une automatiquement).
- Pas de Load Balancer : chaque tâche a une IP publique directe (assign_public_ip = true), pour rester simple et dans les temps. Un ALB serait la solution recommandée en prod.
- Image taguée v1 (jamais latest), conformément à la contrainte du sujet.
- Le Security Group n'autorise que le port 80 en entrée, et la sortie est ouverte pour permettre le pull de l'image depuis ECR.

3.2 Déroulé

J'ai d'abord vérifié que le rôle LabRole existait bien sur mon compte Academy avec `aws iam get-role`. Ensuite `terraform init / plan / apply` pour créer l'infra. Une fois le repo ECR créé, j'ai construit l'image nginx en local avec Docker et je l'ai poussée dessus avant de forcer un nouveau déploiement du service (`aws ecs update-service --force-new-deployment`) pour que les tâches aillent chercher la bonne image.

```
(kali@kali) - [~/Desktop/projet-ecs-k8s]
$ aws ecs describe-services --cluster projet-ecs-k8s-cluster --services projet-ecs-k8s-service --region us-east-1 --query 'services[0].{status:status,running:runningCount,desired:desiredCount}' --no-cli-pager
{
  "status": "ACTIVE",
  "running": 2,
  "desired": 2
}
```

Pour vérifier que ça marchait vraiment, j'ai récupéré l'IP publique d'une des tâches via l'ENI associée et j'ai ouvert la page dans le navigateur.

```
(kali@kali) - [~/Desktop/projet-ecs-k8s]
$ aws ecs list-tasks --cluster projet-ecs-k8s-cluster --region us-east-1 --no-cli-pager
{
  "taskArns": [
    "arn:aws:ecs:us-east-1:721410384633:task/projet-ecs-k8s-cluster/b5a89b54c63d4aa886d65300f7ac7180",
    "arn:aws:ecs:us-east-1:721410384633:task/projet-ecs-k8s-cluster/de5d979b2cd548daaf1a7ea689e2ddea"
  ]
}
```

```
(kali@kali) - [~/Desktop/projet-ecs-k8s]
$ aws ecs describe-tasks --cluster projet-ecs-k8s-cluster --tasks arn:aws:ecs:us-east-1:721410384633:task/projet-ecs-k8s-cluster/b5a89b54c63d4aa886d65300f7ac7180 --region us-east-1 --query 'tasks[0].attachments[0].details' --no-cli-pager
[
  {
    "name": "subnetId",
    "value": "subnet-0e737b2572e431688"
  },
  {
    "name": "networkInterfaceId",
    "value": "eni-0bfbeca044177279d"
  },
  {
    "name": "macAddress",
    "value": "0a:ff:fe:c1:e3:b7"
  },
  {
    "name": "privateDnsName",
    "value": "ip-172-31-30-91.ec2.internal"
  },
  {
    "name": "privateIPv4Address",
    "value": "172.31.30.91"
  }
]
```

IP publique et test de l'URL :

```
(kali@kali) - [~/Desktop/projet-ecs-k8s]
$ aws ec2 describe-network-interfaces --network-interface-ids eni-0bfbeca044177279d --region us-east-1 --query 'NetworkInterfaces[0].Association.PublicIp' --no-cli-pager
"100.27.202.165"
```

Application deployée via ECS Fargate

IPSSI - Mastere Cybersecurite et Cloud Computing

4. Déploiement sur Kubernetes (Minikube)

Le module k8s (modules/k8s) reprend la même logique côté Kubernetes : un namespace dédié projet-app, un ConfigMap avec la page HTML, un Deployment (2 réplicas avec requests/limits CPU et mémoire), un Service ClusterIP, un Ingress, un HorizontalPodAutoscaler (min 2 / max 6 réplicas, cible 50% CPU), et une NetworkPolicy comme garde-fou de sécurité.

4.1 Choix techniques

- J'ai utilisé directement l'image nginx publique (pas besoin de la pousser sur un registre comme pour ECS, ça simplifie beaucoup côté Kubernetes).
- Le garde-fou sécurité choisi est une NetworkPolicy qui restreint le trafic entrant au namespace + au contrôleur Ingress, plutôt que Kyverno (mentionné en exemple dans le sujet) : plus rapide à mettre en place dans le temps imparti, même si Kyverno permettrait un contrôle plus fin (par exemple interdire le tag latest au niveau admission).
- Les requests/limits sur le Deployment sont indispensables pour que le HPA puisse calculer un pourcentage d'utilisation CPU cohérent.

4.2 Déroulé

Les add-ons ingress et metrics-server étaient déjà activés sur mon Minikube (reliquat d'un lab précédent), ce qui m'a fait gagner du temps. Après terraform apply, j'ai vérifié l'état des pods et testé l'accès via l'Ingress avec l'IP de Minikube.

```
(kali@kali)-[~/Desktop/projet-ecs-k8s]
$ kubectl get pods -n projet-app
NAME                                READY   STATUS    RESTARTS   AGE
app-deployment-58ddf6dd9d-kpgv7    1/1     Running   0           161m
app-deployment-58ddf6dd9d-vq6gj    1/1     Running   0           161m
```

Le test d'accès via curl confirme que l'application répond bien à travers le Service et l'Ingress.

```
(kali@kali)-[~/Desktop/projet-ecs-k8s]
$ curl http://$(minikube ip)
<!DOCTYPE html>
<html>
<head><title>Projet Kubernetes</title></head>
<body style="font-family: sans-serif; text-align: center; margin-top: 100px;">
  <h1>Application deployée via Kubernetes (Minikube)</h1>
  <p>IPSSI - Mastere Cybersecurite et Cloud Computing</p>
</body>
</html>
```

5. Automatisation avec Jenkins

Le pipeline est décrit dans un seul Jenkinsfile versionné à la racine du dépôt, avec 4 étapes : Validate (terraform fmt + validate), Plan (terraform plan -out=tfplan), Approve (pause avec validation humaine via l'instruction input), et Apply (terraform apply du plan sauvegardé). Ce même pipeline pilote les deux modules (ecs et k8s) en une seule exécution, puisqu'ils sont tous les deux appelés depuis le main.tf racine.

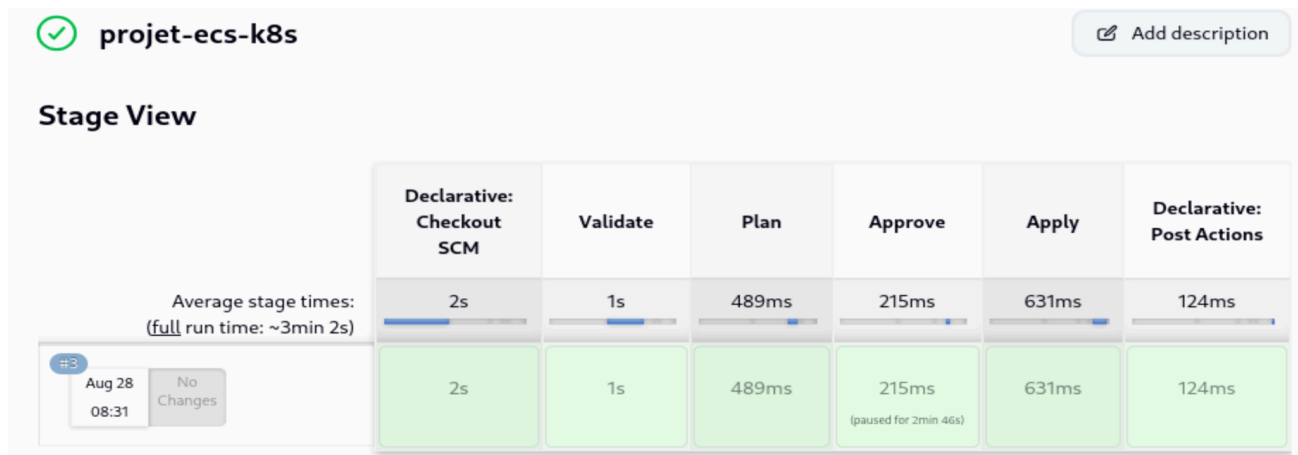
5.1 Configuration

Jenkins tourne en local sur ma VM sous l'utilisateur système jenkins, qui a son propre dossier home séparé du mien. J'ai donc dû copier mes credentials AWS (~/.aws) et mon kubeconfig Minikube (~/.kube et ~/.minikube) vers /var/lib/jenkins, en adaptant les chemins des certificats dans le kubeconfig copié. J'ai aussi dû configurer une clé SSH dans Jenkins pour qu'il puisse cloner mon dépôt GitHub privé, et ajouter la clé d'hôte de github.com dans le known_hosts de l'utilisateur jenkins.

Point important à surveiller : les credentials AWS Academy expirent au bout de 4h. Il faut donc recopier un nouveau jeu de credentials vers /var/lib/jenkins/.aws à chaque régénération de session, sinon le pipeline échoue avec une erreur d'authentification expirée au moment du plan/apply.

5.2 Résultat

Le pipeline s'exécute bien de bout en bout : validation OK, plan sans erreur, pause à l'approbation, puis apply réussi une fois validé manuellement.



La console output confirme le déroulé complet, avec le message SUCCESS à la fin.

```
+ terraform apply -input=false tfplan
[0m[1m[32m
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
[0m
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Déploiement ECS + Kubernetes termine avec succes.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

6. Difficultés rencontrées et solutions

Cette partie liste les principaux blocages que j'ai eus et comment je les ai résolus. Je trouve que c'est la partie la plus intéressante à raconter parce que ça montre la vraie démarche, pas juste le résultat final.

6.1 Fichiers Terraform vides après collage

Plusieurs fois, en collant du code dans VS Code, le fichier ciblé (main.tf ou variables.tf à la racine) s'est retrouvé vide alors que je pensais avoir collé le contenu. Ça venait du fait que VS Code n'affichait que le sous-dossier modules/ au lieu du projet complet, donc je collais parfois dans le mauvais fichier sans m'en rendre compte. Solution : toujours ouvrir le dossier racine du projet dans VS Code (File > Open Folder) et vérifier le chemin affiché en haut de l'onglet avant d'éditer.

6.2 Descriptions de Security Group refusées par AWS

AWS refuse les caractères accentués et les apostrophes dans les champs description des Security Groups (erreur InvalidParameterValue). J'ai dû repasser mes descriptions en anglais sans accents.

6.3 VM à court de RAM pendant le déploiement Kubernetes

Les pods restaient bloqués en ContainerCreating pendant plusieurs minutes sans qu'aucun événement ne remonte dans kubectl describe pod. En vérifiant avec free -h, j'ai découvert que ma VM (4 Go de RAM) avait un swap complètement saturé. J'ai augmenté la RAM allouée à la VM à 8 Go dans VirtualBox, ce qui a réglé le problème immédiatement (les pods sont passés en Running en 2 secondes après redémarrage).

6.4 State Terraform désynchronisé après coupure de la VM

En éteignant la VM pour ajouter de la RAM, j'ai coupé un terraform apply en plein milieu. Au redémarrage, Terraform ne savait plus que certaines ressources Kubernetes existaient déjà côté cluster et essayait de les recréer, ce qui provoquait une erreur already exists. J'ai réimporté la ressource concernée dans le state avec terraform import, ce qui a suffi à resynchroniser.

6.5 Clé GPG du dépôt Jenkins expirée

L'installation de Jenkins échouait avec une erreur de signature (clé expirée depuis mars 2026). Jenkins a changé de clé de signature fin 2025 ; il fallait utiliser l'URL de la clé 2026 (jenkins.io-2026.key) au lieu de celle de 2023 utilisée dans la documentation que j'avais suivie au départ.

6.6 Authentification SSH de Jenkins vers GitHub

Jenkins tourne sous un utilisateur système (jenkins) différent du mien, avec son propre dossier ~/.ssh vide au départ. Il a fallu générer son known_hosts pour github.com et lui fournir une clé privée SSH via les credentials Jenkins pour qu'il puisse cloner mon dépôt privé.

7. Bilan

Ce projet m'a permis de voir concrètement la différence d'approche entre un orchestrateur managé (ECS, où beaucoup de choses sont imposées par la plateforme comme le rôle LabRole) et un orchestrateur qu'on pilote plus finement (Kubernetes, avec ses propres primitives de scaling et de sécurité réseau). La partie la plus formatrice a clairement été le débogage : la majorité du temps n'a pas été passée à écrire du Terraform mais à comprendre pourquoi telle ou telle ressource ne se créait pas, ce qui correspond bien à la réalité du métier.

Pistes d'amélioration si j'avais eu plus de temps : mettre un ALB devant ECS plutôt que des IP publiques directes, utiliser Kyverno côté Kubernetes pour un contrôle d'admission plus strict, et tester le HPA sous charge réelle avec un outil comme hey ou k6.

8. Annexe

Dépôt Git du projet : <https://github.com/Keysan-svg/projet-ecs-k8s>